BACKEND DE UN BANCO DIGITAL

CASO 4 - EAV03

INTEGRANTES

- Atencia Oliva Maria Fernanda
- Echeverry Gonzalez Cristian David
- Molina Domínguez Bryan David
- Pérez Alzate Quelix Xiomara
- Tabares Tobon Michael Stiven
- Céspedes Areiza Camilo Marsel
- Huertas Gonzalez Diana Carolina
- Calle Henao Jhordy
- Ávila Mazo Santiago

Contexto

Los bancos digitales han transformado la forma en que los usuarios interactúan con los servicios financieros. Estas organizaciones operan completamente en línea y dependen de plataformas tecnológicas capaces de gestionar cuentas, transacciones y operaciones financieras de forma segura y escalable.

Construir este tipo de infraestructura requiere sistemas capaces de registrar operaciones financieras, gestionar balances y ofrecer trazabilidad completa de las transacciones. La organización busca desarrollar el núcleo tecnológico de un banco digital que permita administrar cuentas y operaciones básicas.

Arquitectura

Banco Digital — Java 17 · Spring Boot 3 · PostgreSQL

LENGUAJE Y PLATAFORMA

- Java 17 — Lenguaje
- Spring Boot 3 — Framework
- PostgreSQL — Base de datos
- Spring Data JPA — ORM / Queries

DOCUMENTACIÓN Y HERRAMIENTAS

- Swagger UI — Docs API
- Git — Versiones
- GitHub — Repositorio
- Intellij IDEA — IDE

CAPAS DE LA APLICACIÓN

- Controller — Entrada HTTP
- Service — Lógica negocio
- Mapper — Entidad ↔ DTO
- Repository — Consultas BD
- Entity — Tabla en BD
- DTO — Transferencia
- Exception — Manejo errores

HU-01 Registro de Nuevos Usuarios

Como usuario, quiero registrar mi perfil y datos financieros en la aplicación, para que el sistema cree mi identidad digital

POST http://localhost:8080/api/v1/auth/registro

Params | Body | Auth | Headers 4 | Scripts | Docs

JSON

```
1 {
2   "username": "carlos",
3   "password": "carlos123+",
4   "idCliente": 3
5 }
```

201 Created

```
1 {
2   "estado": "ACTIVO",
3   "idUsuario": 3,
4   "username": "carlos"
5 }
```

HU-03 Consulta de Perfil de Clientes

Como cliente activo con sesión iniciada, quiero acceder a mi panel principal (dashboard), para visualizar mis datos personales y el listado de mis productos bancarios activos con sus respectivos saldos.

GET http://localhost:8080/api/profile/3

200 OK | 435 ms | 107 B

Preview | Headers 9 | Cookies | Tests 0 / 0 | → Mock | Console

Preview


```
1 {
2   "fullName": "Carlos Pérez",
3   "identificationNumber": "111111111",
4   "accountNumber": "00030001",
5   "balance": 0.0000
6 }
```

HU-04 Actualizacion del información del Cliente

Como cliente activo con sesión iniciada, quiero modificar mi dirección, teléfono, correo electrónico y fuente de ingresos desde mi perfil, para mantener mi información actualizada y asegurar que las notificaciones del banco lleguen correctamente.

PUT http://localhost:8080/api/v1/clientes/2

Params | Body | Auth | Headers 4 | Scripts | Docs

JSON

```
1 {
2   "telefono": "3167856589",
3   "email": "ana9999@example.com"
4 }
```

200 OK

```
1 {
2   "mensaje": "Tus datos se han actualizado correctamente"
3 }
```

HU-05 Cierre del Producto Financiero

Como cliente activo, quiero tener la opción de solicitar el cierre o inactivación de mi cuenta de ahorros desde la aplicación, para finalizar mi relación contractual con el banco de manera digital.

PATCH http://localhost:8080/api/v1/cuentas/cerrar

Params | Body | Auth | Headers (4) | Scripts

JSON

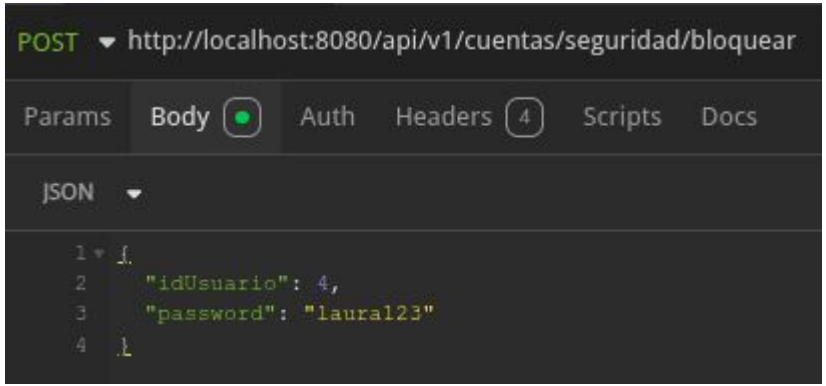
```
1 {
2   "idCuenta": 3,
3   "username": "carlos",
4   "contrasena": "carlos123"
5 }
```

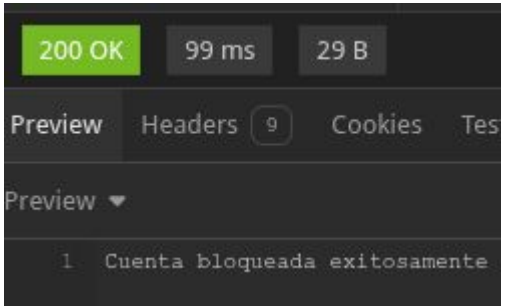
200 OK

```
{
  "numeroCuenta": "00030001",
  "estado": "INACTIVA",
  "mensaje": "El cierre de tu cuenta ha sido realizado exitosamente."
}
```

HU-08 Activación de bloqueo preventivo por el usuario

Como cliente titular de una cuenta bancaria, Quiero tener una opción para bloquear mi cuenta de forma inmediata desde la aplicación, Para evitar retiros o transferencias no autorizadas en caso de pérdida de mi dispositivo o sospecha de fraude.





HU-14: Exposición de movimientos y saldos históricos

Yo como cliente, quiero ver una lista detallada de mis ingresos y egresos con el saldo resultante, para controlar mis finanzas y verificar que no existan cargos desconocidos.

GET http://localhost:8080/api/v1/transacciones/cuenta/1/filtro?fechaInicio=2026-04-07T00:00:00&fechaFin=

Params | Body | Auth | Headers (4) | Scripts | Docs

Form (3)

+ Add | Delete all | Description

idCuenta	1
fechaInicio	2026-04-07T00:00:00
fechaFin	2026-05-07T00:00:00

200 OK


```
1  [
2    {
3      "concepto": "TRANSFERENCIA",
4      "fechaHora": "2026-04-08T00:20:15.56701",
5      "monto": -50000.0000,
6      "saldoResultante": null
7    },
8    {
9      "concepto": "DEPOSITO",
10     "fechaHora": "2026-04-08T00:20:15.560594",
11     "monto": 850000.0000,
12     "saldoResultante": null
13   }
14 ]
```

HU-14: Exposición de movimientos y saldos históricos

Yo como cliente, quiero ver una lista detallada de mis ingresos y egresos con el saldo resultante, para controlar mis finanzas y verificar que no existan cargos desconocidos.

GET http://localhost:8080/api/v1/transacciones/cuenta/1


```
[
  {
    "concepto": "TRANSFERENCIA",
    "fechaHora": "2026-04-07T23:58:45.804454",
    "monto": -50000.0000,
    "saldoResultante": null
  },
  {
    "concepto": "DEPOSITO",
    "fechaHora": "2026-04-07T23:58:45.798447",
    "monto": 850000.0000,
    "saldoResultante": null
  }
]
```

Demo

Backend
https://drive.google.com/file/d/1hjzNTepazql8H-FmhXT2ynyueHnbNCC-/view?usp=sharing

Front
https://drive.google.com/file/d/1nBzvSxgSiAtTnVI2USq8extg_v2GiNZ-/view?usp=sharing

Actualización QA — Sprint / EP-01

Resultados Spring actual:

Estado CI: Passing (workflow ci.yml)

Tests: 51 ejecutados, 0 fallos

Cobertura: Objetivo 80% alcanzado

Alcance: Tests unitarios + mappers + seguridad

Esperado en siguiente Spring:

Sonar Cloud

Mutaciones

Karate por HU

Integracion Back-Front

Obstaculos

0-Falta de claridad en el direccionamiento del proyecto.

1-Definición inicial de la arquitectura.

2-Errores dispersos y respuestas inconsistentes.

3-Definir correctamente relaciones entre Cliente, Usuario y Cuenta.

4-Limitación de tiempo para completar todas las historias.

Acciones

0-Se trabajo con el contexto que traemos de cursos y fabricas anteores para traeles un entregable

1-Se diseñaron previamente diagramas de paquetes y capas, lo que permitió establecer una estructura clara antes de implementar.

2-Se centralizó el manejo de errores usando excepciones personalizadas y un `GlobalExceptionHandler`.

3-Se ajustó el modelo de datos y se utilizaron DTOs para desacoplar las entidades del exterior.

4-Se priorizaron funcionalidades clave y el resto se planificó para próximos sprints.

Aprendizajes

Este sprint nos proporcionó lecciones profundas y significativas, tanto en el aspecto técnico como personal, ya que entendimos que crear un sistema bancario implica más que simplemente programar, sino que conlleva una gran responsabilidad en cuanto a seguridad, organización y calidad. Nos dimos cuenta de la relevancia de planificar antes de codificar, definiendo de manera adecuada la arquitectura y el modelo de datos desde el comienzo para evitar tener que rehacer el trabajo más adelante. Reforzamos nuestro entendimiento sobre prácticas recomendadas como la separación de capas, el empleo de DTOs y la gestión centralizada de errores, entendiendo que estas elecciones afectan directamente la facilidad de mantenimiento y la capacidad de escalar del sistema. Además, enfrentarnos a desafíos como la autenticación mediante JWT nos brindó confianza, disciplina técnica y resiliencia, recordándonos que los errores no son fracasos, sino oportunidades para aprender y evolucionar. Este sprint estableció una base sólida para el proyecto y reafirmó nuestro compromiso con la calidad, el trabajo metódico y el desarrollo continuo como programadores.